

# コンピュータアーキテクチャI

担当：佐々木 敬泰

第14回

浮動小数点の乗算と誤差

## 今日の内容

- 浮動小数点の乗算
- 浮動小数点の丸めと誤差

コンピュータアーキテクチャI

3

## 浮動小数点の乗算(1/2)

- 科学記数法で表現された数値の乗算

$$\begin{aligned} & 1.75 \times 10^2 \times 8.30 \times 10^2 \\ &= (1.75 \times 8.30) \times 10^2 \times 10^2 \\ &= 14.525 \times 10^{(2+2)} \\ &= 14.525 \times 10^4 \\ &= 1.4525 \times 10^5 \end{aligned}$$

正規化

- ゲタばき（バイアス）の影響

$1.75 \times 10^2$ は、 $(1+0.75) \times 10^{(129-127)}$ と考えるため、  
指数部には129が格納されている  
⇒ 指数部を単純に足すのは駄目

5

## 例題(1/3)

- 仮数の精度を4bitとした場合に、  
 $0.5_{10} \times -0.4375_{10}$ を計算せよ。

- 手順

- ステップ1

- $0.5_{10}$ は $1.000_2 \times 2^{-1}$
- $-0.4375_{10}$ は $-1.110_2 \times 2^{-2}$
- ゲタを履かせた表現では、
  - $1.000_2 \times 2^{(126-127)} \times -1.110_2 \times 2^{(125-127)}$

となる。従って指数部は、

$$126+125-127 = 124$$

となる。

7

## シラバス

訂正

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. まとめ ◀ 次週は通常講義  
定期試験

コンピュータアーキテクチャI

2

## 浮動小数点の乗算

コンピュータアーキテクチャI

4

## 浮動小数点の乗算(2/2)

- 指数部

$$\begin{aligned} & 1.75 \times 10^2 \times 8.30 \times 10^2 \\ &= 1.75 \times 10^{(129-127)} \times 8.30 \times 10^{(129-127)} \end{aligned}$$

⇒ 指数部 =  $129+129=258$ は間違い

$$\begin{aligned} & (129-127) + (129-127) + 127 \\ &= 129 + 129 - 127 \\ &= 131 \end{aligned}$$

- 新しい指数部 = ゲタばきさせた指数部  
+ ゲタばきさせた指数部 - 127

6

## 例題(2/3)

- ステップ2

- 仮数を掛け合わせると、  
 $1.000_2 \times 1.110_2 = 01.110000_2$
- 積は $01.110000_2 \times 2^{-3}$ とする。

教科書から若干変更  
しているので注意！

$$\begin{array}{r} 1.??? \times 1.??? \text{なので、} \\ \text{積は1以上になる} \\ \times \begin{array}{r} 1.000 \\ 1.110 \\ \hline 0000 \\ 1000 \\ 1000 \\ \hline 01.110000 \end{array} \end{array}$$

ここが1なら  
正規化が必要

小数点

8

## 例題(3/3)

- ステップ3
  - $01.110000_2 \times 2^3$ は正規数。指数(-3)は127以下、かつ-126以上なのでオーバーフロー、アンダーフローの発生なし
- ステップ4
  - 4bitで丸める
    - $01.110000_2 \Rightarrow 1.110_2$
- ステップ5
  - 乗数、被乗数の符号が異なっているので、積の符号ビットを1にする
    - $\Rightarrow -1.110_2 \times 2^{-3}$

9

## ガード桁、丸め桁

- 有効数字が3桁の場合に、  
 $(2.56_{10} \times 10^0) + (2.34_{10} \times 10^2)$   
 の加算考える
- 加算の場合、まず指数を大きい方の数に揃える必要がある  
 $2.56_{10} \times 10^0 < 2.34_{10} \times 10^2$ より、  
 $2.56_{10} \times 10^0 = 0.0256_{10} \times 10^2$ とする
- 中間結果で精度を落とさないために、有効数字桁の下にガード桁、丸め桁を追加する
- 従って、以下の加算式になる  
 $(0.0256_{10} \times 10^2) + (2.34_{10} \times 10^2)$

11

## 丸め

- コンピュータで扱えるビットは有限、かつ事前に決まっている
  - 仮数のビット幅を超える値をレジスタに格納するには、情報の一部を切り捨てる必要がある
  - $3.14159265 \Rightarrow 3.14$  (3桁に丸める)
- IEEE754では4つ規定されている
  - 常に切り上げ (+ $\infty$ 方向への丸め)
  - 常に切り下げ (- $\infty$ 方向への丸め)
  - 切り捨て (0方向への丸め)
  - 最も近い偶数への丸め

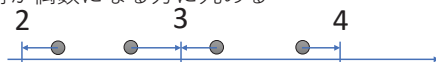
13

## 最も近い偶数への丸め(1/2)

- 既定されたビットに収めるために、データを切り捨てるのは仕方ない (必ず誤差は生じる)

↓ しかし

- 実際の数値に近い値に丸める方が望ましい
- そこで、
  - 最も近くの表現できる値へ丸める
  - 表現可能な2つの値の中間の値であつたら、一番低い桁が偶数になる方に丸める



- 四捨五入と同じ？

15

## 丸めと誤差

10

## ガード桁、丸め桁 (有効数字3桁)

- ガード桁、丸め桁なし
 

|        |   |     |
|--------|---|-----|
| 0      | 0 | 2   |
| +      | 2 | 3 4 |
| 2. 3 6 |   |     |
  - ガード桁、丸め桁あり
 

|        |   |   |   |     |
|--------|---|---|---|-----|
| 0      | 0 | 2 | 5 | 6   |
| +      | 2 | 3 | 4 | 0 0 |
| 2. 3 6 |   |   | 5 | 6   |

有効桁      四捨五入
- 誤差
- ↓
- 2.37
- $2.37_{10} \times 10^2$

12

## 丸めモード

- 常に切り上げ (+ $\infty$ 方向への丸め)
  - $1.5 \Rightarrow 2$
  - $-1.5 \Rightarrow -1$
- 常に切り下げ (- $\infty$ 方向への丸め)
  - $1.5 \Rightarrow 1$
  - $-1.5 \Rightarrow -2$
- 切り捨て (0方向への丸め)
  - $1.5 \Rightarrow 1$
  - $-1.5 \Rightarrow -1$
- 最も近い偶数への丸め
  - ⇒次スライドで説明

14

## 最も近い偶数への丸め(2/2)

- 表現可能な2つの値の中間の値であつたら？
  - 0.5を整数に丸めると0? 1?
    - 0とすると、 $\text{int}(2.5) + \text{int}(3.5) = 5$
    - 1とすると、 $\text{int}(2.5) + \text{int}(3.5) = 7$  (四捨五入)
  - 一番低い桁が偶数になる方に丸める



真に中間値だった場合、片方に丸めると誤差が蓄積されていく。そのため、(多分ランダムであろう)上位桁によって丸める方向を決める

最も近い偶数への丸めの場合、 $\text{int}(2.5) + \text{int}(3.5) = 2 + 4 = 6$

16

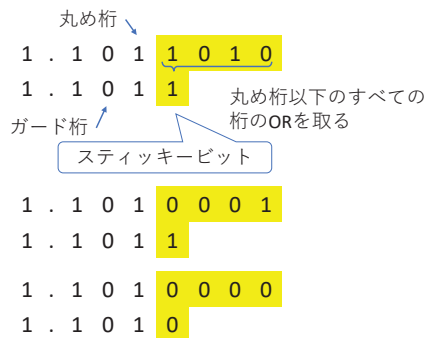
## 最も近い偶数への丸めと中間値

- $2.34_{10} \times 10^2 + 5.01_{10} \times 10^{-1}$ を考える
- 指数を揃えると仮数は  $2.34 + 0.0050 = 2.3450$   
ガード桁(5)、丸め桁(0)
- 最も近い偶数への丸めをすると、有効桁の最下位(4)は偶数なので、2.34になる
- しかし、5.0ではなく5.01なので、  
 $2.34 + 0.00501 = 2.34501$   
は2.35に丸める方が望ましい

17

## スティッキービットのイメージ

- スティッキービットは、丸め桁以下の代表値



19

## 再掲 知らないとうなる？

- コンピュータアーキテクチャなんて知らなくてもコンピュータ使えるよ？
  - 『使うだけ』なら使える  
⇒それなら情報系出身の人材じゃなくてもいい
  - 『動くプログラムを書くだけ』ならコンピュータアーキテクチャを知らなくても書ける  
⇒性能は？ 安定性は？ 常に稼働条件を満たしてる？ 移植性は？
- 情報系出身学生の強みは？

コンピュータアーキテクチャ

21

## 再掲 事例A(2/3)

```
• Cのプログラム
#include <stdio.h>

int main(){
    float x, y; /* 浮動小数点の変数を宣言 */

    x = 0.125; /* xに0.125を代入 */
    y = 4000000.0; /* yに4000000.0を代入 */

    printf("x = %f\n", x); /* xを表示 */
    printf("y = %f\n", y); /* yを表示 */
    printf("(x+x)+y = %f\n", (x+x)+y);
    printf("x+(x+y) = %f\n", x+(x+y));
    return 0;
}
```

コンピュータアーキテクチャ

23

## スティッキービット

- 有効桁以下が無限ならより正確な近似が可能  
↓ しかし
- 実際にはハードウェアリソースは有限
- ガード桁、丸め桁に加え、スティッキービットを導入する
- スティッキービットは、
  - 丸め桁より下の桁の代表値
  - 丸め桁の更に一つ下の1bit
  - 最初は0だが、一度1になると0にはならない

18

## スティッキービットの効果

- ガード桁(G)、丸め桁(R)のみ

|        | 有効桁         | G | R |                       |
|--------|-------------|---|---|-----------------------|
| 元の値    | 1 0 0 0 1 0 |   |   | $100010_2 \times 2^0$ |
| 1つ右シフト | 0 1 0 0 0 1 |   |   | $10001_2 \times 2^1$  |
| 2つ右シフト | 0 0 1 0 0 0 |   |   | $1000_2 \times 2^2$   |
| 3つ右シフト | 0 0 0 1 0 0 |   |   | $100_2 \times 2^3$    |

- スティッキービットあり

|        | 有効桁           | G | R | S |                        |
|--------|---------------|---|---|---|------------------------|
| 元の値    | 1 0 0 0 1 0 0 |   |   |   | $1000100_2 \times 2^0$ |
| 1つ右シフト | 0 1 0 0 0 1 0 |   |   |   | $100010_2 \times 2^1$  |
| 2つ右シフト | 0 0 1 0 0 0 1 |   |   |   | $10001_2 \times 2^2$   |
| 3つ右シフト | 0 0 0 1 0 0 1 |   |   |   | $1001_2 \times 2^3$    |

$1000_2 \times 2^3$ より大きいという情報が残る

20

## 再掲 事例A(1/3)

- 結合則の実験
- $(A + B) + C = A + (B + C)$ ?  
 $0.125 + 0.125 + 4000000.0$ を考える  
  
 $(0.125 + 0.125) + 4000000.0$ と  
 $0.125 + (0.125 + 4000000.0)$ は等しい？

コンピュータアーキテクチャ

22

## 再掲 事例A(3/3)

- 実験  

```
% gcc -o add_error add_error.c
% ./add_error
x = 0.125000
y = 4000000.000000
(x+x)+y = 4000000.250000
x+(x+y) = 4000000.000000
```

結合則は成り立たない(?)

➡ 浮動小数点の授業を理解できれば  
納得できる結果

コンピュータアーキテクチャ

24

## 事例Aの原因

- $\bullet$   $0.125 + \underline{(0.125 + 4000000.0)}$
- $\bullet$   $1.25 \times 10^{-1} + 4.00 \times 10^6$ 
  - $\bullet$   $1.25 \times 10^{-1}$   
 $1.000000000000000000000000_2 \times 2^{-3}$
  - $\bullet$   $4.00 \times 10^6$   
 $1.111010000100100000000000_2 \times 2^{21}$
- 指数を揃えると、
 

GRS桁

|                                                    |  |                       |
|----------------------------------------------------|--|-----------------------|
| $00000000000000000000000000000000_2 \times 2^{21}$ |  | $100_2 \times 2^{21}$ |
| $+ 111101000010010000000000000000_2 \times 2^{21}$ |  | $100_2 \times 2^{21}$ |
| <hr/>                                              |  |                       |
| $111101000010010000000000000000_2$                 |  | $100_2 \times 2^{21}$ |

➡  $4000000.0_{10}$

最も近い偶数への丸め

25

## 練習

- IEEE754単精度を用いて加算することを考える
  - ここで、
    - $X=0.125$
    - $Y=4000000.25$
- としたとき、
- A)  $(X+X)+Y$
- B)  $X+(X+Y)$
- の結果がどうなるか考えよ

27

## 今日のまとめ

- 浮動小数点表現では、大きな数と小さな数を足すと危険！

⇒数値計算系では非常に重要！！

29

## 事例Aのアレンジ版

- $0.125 + 4000000.25$ なら？
  - $1.25 \times 10^{-1} + 4.00000025 \times 10^6$ 
    - $4.00000025 \times 10^6$
- $$1.111010000100100000000001_2 \times 2^{21}$$
- 指数を揃えると、
- $$\begin{array}{r} 0000000000000000000000000000100_2 \times 2^{21} \\ + 1111010000100100000000001000_2 \times 2^{21} \\ \hline 1111010000100100000000001100_2 \times 2^{21} \end{array}$$
- 最も近い偶数へ丸めると、
- $$1111010000100100000000010_2 \times 2^{21}$$
- ➡  $4000000.5_{10}$

26

## 練習の答え

$$x+y = 4000000.500000$$

28